String handling in Java :

What is a String ?

* A string is basically **sequence of characters**, these characters can be anything like alphabets, digits, special symbols which could be enclosed with double quotes.

* In Java, string is basically an array of characters with double quotes; java has provided a predefined class called **String** available in java.lang package.

Example : String s1 = "Java";
String s2 = "Java";
String s3 = "Java";

So, the conclusion is : String is a collection of alpha-numeric character.

How many ways we can create String object in java :

* There are 2 ways to create String object in Java :

1) By using literal notation
Example : String s1 = "Java"; //Since JDK1internally calling intern() method

2) By using new keyword
Example : String s2 = new String("Hibernate");

3) By using character array
char ch[] = {'M', 'K', 'S'};

//Program :

```
public class
{
String s1 = "Java";
String s2 = new String("Hibernate");
char ch[] = {'M','K','S'};
}
System.out.
System.out.println(s1);
System.out.println(s2);
System.out.println(new String(ch));
```

Immutability in String :

* Strings are immutable (un-changed OR un-modifiable) in java that means once we create the string object it is not modifiable.

* We have various cases for immutability of the String :

Case 1 :

```
String s1 = "Java";
String s2 = "Java";
System.out.
System.out.
System.out.println(s1 == s2);
```

HEAP MEMORY

Reference can create a String object by using literal notation, in this case JVM will check for any special memory area called SCP (Pool) and if it is not present then JVM will create it.

SCP => Intern() method

JVM -> Internally JVM will refer to capital JVM

Case 2 :

```
String s1 = "Java";  //Pool area
System.out.println(System.identityHashCode(s1));
System.out.
System.out.
```

HEAP MEMORY

Case 3 :

```
String s1 = "Java";
String s2 = s1.toUpperCase();
System.out.
System.out.
System.out.println(s1 == s2);  //false
```

HEAP MEMORY

Case 4 :

```
String s1 = "Java";
String s2 = "Java";
System.out.
System.out.
System.out.println(s1 == s2);  //true
```

HEAP MEMORY

Case 5 :

```
String s1 = new String("India");
System.out.
System.out.
System.out.println(s1==s2);  //false
```

SCP one does not contain duplicate value.

HEAP MEMORY

Immutability in String :

In Java String Objects are immutable means unchanged or, whenever we create a String object in java it can't be modifiable.

Strings literals are created in a very special memory of RAM called String Constant Pool(SCP) [JDK 1.8 Metaspace] and it is not available in other memory.

String once created can't be modifiable.

Facts about String and memory :

HEAP MEMORY

In Java Whenever we create a new String object by using String literal, first of all JVM will verify whether the String we want to create is pre-existing (already available) in the String Constant pool or not.

If the String is pre-existing (already available) in the String Constant pool then **JVM WILL NOT CREATE ANY NEW STRING OBJECT. INSTEAD IT WILL ASSIGN THE REFERENCE TO THE EXISTING OBJECT.** (Pool Reuse)

Note : In Pool area we can't have duplicate String object.

Why strings objects are immutable in java ?

HEAP MEMORY

HashMap<String,Integer> hm = new HashMap<>();

Key => HashMap

It is recommended to make the key as immutable because if key is mutable then Hashcode value will change.

* * Why Strings are immutable :

As we know a String object in the String constant pool can be refer by multiple reference variables. If any of the reference variable will modify the value of the string object then it will affect all the other reference variables pointing to same String object in spite the original value, since they have defined earlier as above in the diagram.

While working with Collection Framework, String & Wrapper classes we are using as a HashMap key so it is strongly recommended that HashMap key must be immutable. It also is recommended to make Wrapper classes are immutable.